

SDD-WIND-001 · глибокий архітектурний аудит

Wind Engine

Чи це узагальнений 2D-рушій, чи архітектура навколо поточної гри?

C++23 · статична бібліотека `engine` · репозиторій Wind

Висновок ґрунтується на коді в `include/engine/`, `src/`, CMake і тестах — не на припущеннях з документації.

Короткий висновок

Wind — невеликий вбудований 2D-рушій на C++23 (CMake-таргет і простір імен `engine`), який ігри споживають як `git-submodule`. Цей аудит побудований на публічних заголовках у `include/engine/`, реалізації в `src/`, системі збірки, тестах і SDD. Твердження нижче — з перевіреного коду, а не з домислів за документами.

Головне: це не гра, яка випадково прикидається рушієм. Ігрові типи (`Player`, `Enemy`, `Weapon`) у код рушія не протікають. Межа `engine/game` навколо `IGame`, `ECS`, `GUID`-асетів і правила «гра ніколи не викликає `OpenGL`» — справжня, і її варто берегти.

Проблема в іншому. Архітектуру **обмежує поточний продукт**: `desktop-overlay companion` (прозоре `always-on-top` вікно, `click-through`, `drag`-регіони, вторинні вікна налаштувань) плюс аудіо-політика у формі `Lumenwake`. Ці вимоги реалізовані як поведінка головного циклу рушія, а не як опційний `window-backend`. Саме це — головний довгостроковий ризик.

1. Підсумкові оцінки

Вимір	Оцінка
Архітектура	7 / 10
Якість абстракцій	6.5 / 10
Модульність	6 / 10
Розділення <code>engine / game</code>	7.5 / 10
Стійкість до змін вимог	6 / 10
Розширюваність	6 / 10
Дизайн C++	7 / 10
Ownership / lifetime	6.5 / 10
Тестованість	8 / 10
Продуктивнісна архітектура	6.5 / 10

У Wind є цілісна форма v1: один `ecs::World`, `command-buffer` рендер, `GUID`-асети, UI як `XML+CSS+MVVM`, фіксований крок симуляції і `headless Host` для тестів. Публічні заголовки вільні від `SDL/glad/NanoVG/spdlog`. Це правильний рівень амбіції для малого 2D-рушія.

Проблема здоров'я — не відсутність «модного ECS» і не брак інтерфейсів. Проблема в тому, що **композиція амбієтна** (`World::ctx<T>()`), **кадр — закритий `enum` фаз, віконний хост — `god object`**, а **`desktop-overlay` поведінка вварена в головний цикл**. Рендер-інтерфейси існують, але `EngineRuntime` жорстко створює `OpenGL` і на `init` жадібно завантажує весь каталог UI-картинок. Такі рішення буде дорого розвертати, коли на цей коміт сядуть інші ігри.

2. Топ-10 архітектурних проблем

2.1. Overlay-поведінка — це ядро циклу, а не опційна політика вікна

- **Серйозність:** Critical
- **Де:** `EngineRuntime::tick_loop / reentrant_tick (src/core/engine_runtime.cpp)`; `click-through` і `Win32 WndProc` у `WindowSystem (src/render/opengl/window_system.cpp)`; `IWindowControl` (`include/engine/core/window_control.h`); SDD §21

- **Проблема:** Click-through, drag-як-titlebar, прозоре always-on-top, опитування OS-курсора щотику і реентрантний тік всередині `SDL_PollEvent` під час `Win32 modal move/size` реалізовані як lifecycle рушія, а не як overlay-плагін гри.
- **Чому це важливо:** Функція кадру тепер має два шляхи виконання з різною семантикою flush подій. Звичайна fullscreen-гра платить за overlay-крайові випадки, а баги overlay (застарілий `MouseConsumed`, конфлікт `drag-region` із `WS_EX_TRANSPARENT`) уже змушували латати рушій.
- **Поточне припущення:** Первинне вікно може бути безрамковим прозорим desktop overlay, який має симулювати навіть коли OS у циклі `move/size`.
- **Що викриє:** Звичайна гра, унікальний fullscreen-рендер, або будь-яка не-Windows платформа, де цей код — мертвий вантаж, але все одно формує API.
- **Напрямок:** Залишити фічі, але трактувати їх як window-style політику за `WindowStyle / IWindowControl`. Overlay-полінг і reentrant tick вмикати лише коли цей стиль увімкнений.
- **Пріоритет:** **FIX WHEN TOUCHING THIS SYSTEM** (працює; витягати зараз — зайвий churn). Не додавати нові overlay-спецвипадки в `tick_loop` без об'єкта політики.

2.2. `World::ctx<T>()` — необмежений service locator

- **Серйозність:** High
- **Де:** `World::ctx` у `include/engine/ecs/world.inl`; споживачі в `src/ecs/systems.cpp`, `src/core/host.cpp`, `src/core/engine_runtime.cpp`, `include/engine/ui/canvas.h`
- **Проблема:** Будь-який тип можна default-construct у світ при першому доступі. Тут живуть `Time`, `ApplicationState`, `ActiveCamera`, `WindowSize`, `WindowSizes`, `UiPointer`, `UiPointers`, `MouseConsumed`, `PhysicsOverlapState`, `EngineSystemsRegistered` і кожен `Events<T>`. SDD забороняє service locator (`Engine::get_audio()`); це той самий патерн під красивішою назвою.
- **Чому це важливо:** Приховані залежності, неявний порядок ініціалізації, немає власника «хто має право писати сюди». Паралелізм, мережа і ізольовані тести системи стають важкими, бо світ — глобальний мішок.
- **Поточне припущення:** Один main-thread світ; ресурси з'являються магічно; один reader на тип події через `ctx<EventCursor<T>>()`.
- **Що викриє:** Другий світ, серверна симуляція без UI-ресурсів, дві системи з незалежними `EventReader`, спроба ганяти системи в іншому потоці.
- **Напрямок:** Залишити `ctx` для *малого явного* набору (`Time`, `ApplicationState`, `Events<T>`). Не додавати нові UI/window-синглтони. Краще передавати `WindowSize` в `camera/UI` аргументом (камера вже бере `ui::WindowSize` за значенням — це правильна форма; проблема в зберіганні лише в `ctx`).
- **Пріоритет:** **FIX WHEN TOUCHING THIS SYSTEM** для нових ресурсів; **DON'T FIX** тотальне видалення `ctx`.

2.3. Рендер-інтерфейси — не шов під бекенд

- **Серйозність:** High
- **Де:** `EngineRuntime::Impl` (`src/core/engine_runtime.cpp`); `OpenGLRenderBackend::execute_draw_mesh` (`src/render/opengl/opengl_backend.cpp`); `OpenGLCanvas::init` (`src/render/opengl/opengl_canvas.cpp`); `IMesh / IShader / ITexture` (`include/engine/render/graphics.h`)
- **Проблема:** `IRenderBackend`, `IGraphicFactory` і `ICanvas` існують, але хост напряду конструює `OpenGLFactory / OpenGLRenderBackend / WindowManager`. Виконання робить `dynamic_pointer_cast` на `OpenGLShader / OpenGLMesh / OpenGLTexture`. Імена `uniform` (`uModel`, `uView`, `uProjection`, `uColor`, `uTexture`) і один `albedo`-слот зашиті. `IMesh / IShader / ITexture` — порожні віртуальні деструктори.
- **Чому це важливо:** Замінити OpenGL — це не «імплементуй інтерфейс». Це переписати хост, касти бекенда, конвенцію шейдерів, модель матеріалу, `NanoVG painter` і `bootstrap`

вікна/GL.

- **Поточне припущення:** Одна GL-родина (GL 3.3 / GLES3 / WebGL2) з NanoVG UI і XML-шейдерами з цими uniform.
- **Що викриє:** Vulkan / Metal / DirectX, compute-шлях або навіть software-рендер для dedicated server present.
- **Напрямок:** Або чесно вважати GL+NanoVG *єдиним* рендером на роки (це валідно), або змусити EngineRuntime приймати platform/render модуль замість конструювання OpenGL-типів. Поки що не плодити нові порожні GPU-інтерфейси.
- **Пріоритет:** **FUTURE CONSIDERATION**, якщо GL — свідомий бекенд; **FIX NOW** лише якщо другий бекенд уже запланований.

2.4. Жадібна резидентність: кожен font/UI image з каталогу вантажиться на init

- **Серйозність:** High
- **Де:** Engine<GameT>::init (include/engine/core/engine.h); кеш AssetsDb (src/resources/assets_db.cpp)
- **Проблема:** Після завантаження каталогів init обходить кожен Font і кожен Texture/UIImage і заливає їх у primary canvas / NanoVG atlas. AssetsDb кешує назавжди; API unload/evict немає.
- **Чому це важливо:** Стрімінг світу, великі атласи й опційний DLC архітектурно заблоковані. Пам'ять росте з розміром каталогу, а не з working set. Вторинні вікна потім *повторно програють* шрифти в другий атлас (EngineRuntime::tick_loop).
- **Поточне припущення:** Весь cooked-каталог вміщається в RAM і GPU на старті; асети незмінні протягом процесу.
- **Що викриє:** Контентно важка гра, стрімінг рівнів, runtime-пакети мов/шрифтів.
- **Напрямок:** Вантажити GPU-ресурси на перший get<T>; пізніше додати явний release/clear_cache. Припинити preload усіх UI-картинок в init, якщо canvas на них не посилається.
- **Пріоритет:** **FIX NOW** для preload-циклу в init (мала зміна, велика майбутня ціна). Unload API — **FUTURE CONSIDERATION**.

2.5. Закритий schedule + register_engine_systems як фіксований пайплайн

- **Серйозність:** High
- **Де:** include/engine/ecs/schedule.h; register_engine_systems (src/ecs/systems.cpp)
- **Проблема:** Phase — закритий enum із 7 значень. Системи рушія завжди реєструються: physics, UI input, splash timers, bind, audio, world render, UI render. Ігри можуть додавати лише Phase::Game. Host викликає register_engine_systems(world) з порожніми deps, тож render/audio/bind тихо по-оп, але physics і UI input все одно біжать.
- **Чому це важливо:** Інший цикл (покроковий, серверний тік без UI, render-before-sim, два фізичні світи) вимагає правити enum рушія і функцію реєстрації. Старіння splash живе в Phase::Input, бо там був вільний слот.
- **Поточне припущення:** Кожна гра хоче саме цей 2D realtime пайплайн.
- **Що викриє:** Симуляційний сервер, хост інструмента/редактора, гра, якій не потрібна AABV-фізика.
- **Напрямок:** Залишити дефолтний пайплайн, але зробити системи рушія opt-in (register_physics, register_ui, ...) або не реєструвати їх, коли deps порожні. Нова фаза має бути рідкістю; запихати сторонню роботу в Input треба припинити.
- **Пріоритет:** **FIX WHEN TOUCHING THIS SYSTEM**

2.6. EngineRuntime — god object

- **Серйозність:** High
- **Де:** `src/core/engine_runtime.cpp` (~640 рядків); шаблон `init` у `include/engine/core/engine.h`
- **Проблема:** Один клас володіє SDL video, multi-window GL, опитуванням інпуту, staging APK на Android, Emscripten RAF, завантаженням шрифтів/картинок, overlay click-through, реентрантністю modal-loop, фіксованим годинником, update аудіо і колбеками IGame. `Engine<GameT>::init` — 120-рядкова header-only процедура, яка ще й знає каталоги, шрифти, іконки та реєстрацію систем.
- **Чому це важливо:** Будь-яка зміна хоста (нова платформа, headless, інший present) править ту саму функцію. Тести не можуть інстанціювати віконний хост без усієї купи (і не інстанціюють: `engine_tests` ніколи не викликає `Engine::run`).
- **Поточне припущення:** Одна віконна процес-модель з опційними web/android ifdefs.
- **Що викриє:** Dedicated server, тік редактора, другий рендерер.
- **Напрямок:** Відокремити «platform loop» від «window/GL» від «game tick». `Host::tick` уже є правильним витягом; віконний `run` має викликати той самий `tick`, а не паралельну копію.
- **Пріоритет:** **FIX NOW** для дублювання `Host/EngineRuntime tick`; подальші розрізи — коли чіпаєте платформи.

2.7. Два цикли кадру, що роз'їжджаються (Host vs EngineRuntime)

- **Серйозність:** Medium
- **Де:** `src/core/host.cpp`; `EngineRuntime::tick_loop / reentrant_tick`
- **Проблема:** Два цикли роблять `flush` → `input/lifecycle` → `FixedStepClock` → `audio update` → `on_fixed_update` → `on_update` → `present`. Вони вже розійшлися: `Host` ніколи не викликає `IGame::on_draw` (тест `Host.FakeCanvasDraw` очікує `draw_calls == 0`); віконний шлях презентує через `WindowManager::draw_all`; `Host` не політь OS-події; `reentrant_tick` пропускає `flush_events` і `poll_events`. `ui::begin_frame` викликається і з хост-циклу, і з `run_input`.
- **Чому це важливо:** Задokumentований `lifecycle IGame (on_draw)` — брехня. Майбутні фікси циклу сядуть в один шлях і не в інший.
- **Поточне припущення:** Тести використовують `Host`; ігри — `EngineRuntime`; їм достатньо бути «достатньо схожими».
- **Що викриє:** `Pause`, `splash`, аудіо чи інтерполяція, які мають збігатися в тестах і в шипінгу.
- **Напрямок:** Одна реалізація `tick`. `on_draw` треба або викликати, або прибрати з `IGame`.
- **Пріоритет:** **FIX NOW** для контракту `on_draw`; уніфікація тіків — **WHEN TOUCHING** цикл.

2.8. Інверсії шарів: camera/UI, commands/UI, хаб ECS-систем

- **Серйозність:** Medium
- **Де:** `include/engine/ecs/camera.h` інклудить `include/engine/ui/canvas.h`; `include/engine/render/commands.h` тягне UI-типи документів; `src/ecs/systems.cpp` інклудить `audio`, `UI`, `render`, `splash`, `physics`, `camera`
- **Проблема:** Аспект камери визначено через `ui::WindowSize`. Команди малювання несуть `UiDocument*`. Системи рушія — одна `translation unit`, яка знає кожен модуль. `WindowSize/WindowSizes` і `UiPointer/UiPointers` дубльовані як «primary vs others», щоб не ламати старі call site (`include/engine/ui/canvas.h`).
- **Чому це важливо:** Не можна міркувати про ECS чи камеру без UI. Вторинні вікна — `compatibility-форк`, а не єдина таблиця `WindowId` → `size`.
- **Поточне припущення:** Одне первинне вікно — справжня гра; зайві вікна — лише UI overlay/налаштування.

- **Що викриє:** Рендер світу в другому вікні або headless-сим, якому все одно потрібна камера.
- **Напрямок:** Перенести `WindowSize` у `core` (не в `ui`). Одна мапа за `WindowId`. `CmdDrawUI` має бути непрозорим `UI submit`, а не вказівниками на `AST` розмітки.
- **Пріоритет:** **FIX WHEN TOUCHING THIS SYSTEM**

2.9. Ownership загалом RAII, але кілька API ховають lifetime

- **Серйозність:** Medium
- **Де:** `EngineSystemDeps`, захоплений у `closure` систем `World` (`src/ecs/systems.cpp`); `InputSystem::world_` (`include/engine/core/input_system.h`); сирі вказівники в `CmdDrawUI` (`include/engine/render/commands.h`); `shared_ptr` у `Renderable/IMaterial`; `Boost.DI shared_ptr` синглтони (`include/engine/core/engine.h`)
- **Проблема:** GPU-об'єкти розділяються між кешем `AssetsDb`, компонентами `Renderable` і `command buffer` (`refcount` як ідентичність для `sort`). UI-команди тримають вказівники в `UiInstance`, валідні лише до наступного `bind/clone`. Системи рушія зберігають `raw pointer` на об'єкти, якими володіє `Engine`. `ViewModel::property` зберігає `raw pointer` на члени `Bindable<T>`.
- **Чому це важливо:** Зруйнувати вікно, зробити `dispose` рушія, поки `World` ще живий, або клонувати UI-документ посеред кадру — легко зламати. `shared_ptr` на кожен `mesh draw` — також продуктивнісний/архітектурний запах.
- **Поточне припущення:** `Main thread`, один `Engine`, `draw` у тому ж кадрі, що й `push` команд, `Bindable` переживають реєстрацію `ViewModel`.
- **Що викриє:** Стрімінг (знищення сутностей із GPU-посиланнями), `undo` в редакторі, зберігання `World` після `Engine::dispose`.
- **Напрямок:** Хендли (`AssetId` уже є) на `Renderable` замість `shared_ptr<IMesh>`; резолвити на `render`. `CmdDrawUI` можна лишити `same-frame`, але не класти типи UI `AST` у публічний `command-заголовок`.
- **Пріоритет:** **FIX WHEN TOUCHING** `render/assets`; **DON'T FIX** `raw pointer` у `ViewModel` (це реєстрація членів).

2.10. Три філософії обробки помилок одночасно

- **Серйозність:** Medium
- **Де:** `AssetsDb::get` → `IFatalError::report`, потім `std::abort` (`src/resources/assets_db.cpp`); `World::get` — `assert` (`world.inl`); `Engine::init` повертає `bool`; невідома CSS-властивість — `warn`; відсутній `{binding}` — `fatal`; відсутня камера в `render` — `fatal` і `return`
- **Проблема:** Відновлюваний `std::expected` (`try_get`, парсинг каталогу/XML) стоїть поруч із `fatal`-діалогами, `abort`, `assert` і тихими `skip` (`run_audio` продовжує, якщо `deps` `null`; UI вторинного вікна пропускається, якщо буфера немає).
- **Чому це важливо:** Програміст гри не може передбачити, чи помилка — діалог+вихід, `abort` у тесті, чи порожній кадр.
- **Поточне припущення:** Відсутній `cooked-асет` завжди помилка програміста; тести підміняють `IFatalError`.
- **Що викриє:** Опційний контент мода (частково покрито `try_get`) vs помилки шипінг-даних, які не повинні `abort` після діалога.
- **Напрямок:** `get` може бути `fatal`, але `abort` після `report` зайвий, якщо вже викликано `quit()`. Не `silent-skip` помилки програміста в `render`. `expected` залишити для I/O.
- **Пріоритет:** **FIX WHEN TOUCHING THIS SYSTEM**

3. Ігроспецифічні припущення

Це місця, де **архітектура рушія** кодує конкретний продукт, а не просто конкретну реалізацію.

Місце	Поточне припущення	Чому це специфіка гри	Майбутня проблема	Серйозність
<code>IWindowControl, WindowStyle::{transparent, always_on_top, borderless}, click-through y WindowSystem</code>	Первинне вікно може бути desktop overlay; кліки по порожньому UI проходять на робочий стіл	SDD §21 описує «desktop-overlay companion game». Більшості 2D-ігор не потрібні Win32 <code>WS_EX_TRANSPARENT</code> чи drag-як-caption	Складність циклу, Windows-only поведінка в публічному API; інші ігри успадковують overlay-баги	High
Полінг курсора в <code>tick_loop + reentrant_tick</code>	Overlay click-through вбиває OS mouse-move; Win32 modal drag не повинен заморожувати сим	Специфіка overlay + Win32 modal loops	Два шляхи тіка; правила flush подій, яких гра не бачить	High
<code>SplashScreen::image = builtin::splash_window_image.h</code>	Дефолтний splash — брендинг Wind	Ідентичність продукту рушія в дефолті ігрового контракту	Кожна гра з дефолтним struct покаже splash Wind	Low
Аудіо <code>kSfxPoolSize = 12, kMusicSlotCount = 2, A/B fade</code> (SDD «Lumenwake-shaped»)	Два музичні стеми і малий SFX-пул	Політика скопійована з мікзера попередньої гри	Музично/VO-важка гра впирається в тихий drop (<code>play_sfx skip</code> , коли пул повний) без точки розширення, крім нового API	Medium
<code>run_render + SDD §21.1</code>	Світовий Renderable малюється лише в primary window; вторинні вікна — тільки UI	Продукт overlay + вікно налаштувань	Split-screen чи вікно карти не зможе перевикористати render-систему	Medium
<code>WindowSize</code> vs <code>WindowSizes</code> , <code>UiPointer</code> vs <code>UiPointers</code>	Primary лишається синглтоном, щоб старі ігри не змінювати	Сумісність із call site першої гри, не правило домену	Подвійні шляхи читання назавжди; легко оновити один і пропустити інший	Medium
<code>android_back_quits()</code> завжди true	Android Back завжди виходить	Багато ігор хочуть Back = pop UI	Не можна скласти стек екранів без форка lifecycle	Low
<code>ControlKind::Gamepad*</code> без обробників	Геймпад є в моделі	Незавершене узагальнення, не потреба поточної гри	Викликач думає, що bind геймпада працює	Low
Фізика як <code>Phase::Physics AABB/коло</code> , без резолюції	«Physics» означає 2D overlap-події по XY	Нормально як *модуль*, але займає єдину physics-фазу	Гра з Box2D усе одно ганятиме цю систему, якщо не обійде <code>register_engine_systems</code>	Medium
Набір UI (<code>Canvas/Stack/Label/Button/Image/ItemsControl</code>)	HUD/меню як малий WPF-піднабір	UI-kit продукту, не онтологія віджетів рушія	Радикально інший UI (дієгетичний, ImGui, текст) мусить боротися з кітом або обходити його	Low (прийнятно)
Немає <code>Player/Enemy/Weapon</code> в <code>include/</code> чи <code>src/</code>	—	Пошук виконано; лише приклад у SDD з сутністю Player	Немає	н/д

Не класифіковано як ігроспецифічне (конкретна реалізація — нормально): OpenGL 3.3, WAV-only міксер, самописний ECS, TOML .meta, NanoVG, 60 Гц kFixed, відсутність parenting трансформів, відсутність 3D-освітлення. Це scope v1.

4. Карта архітектури

4.1. Логічні модулі

Application	IGame / GameBase (репозиторій гри)
Core host	Engine<GameT> -- Boost.DI -- EngineRuntime (SDL/GL/вікна) Host (headless tick) InputSystem, Time, FixedStepClock, log, IFatalError, Platform
ECS	World, Entity, View, Events<T>, Schedule/Phase register_engine_systems -> physics, ui input, splash, bind, audio, render, ui render
Resources	AssetsDb, CookedCatalog, asset_codegen / asset_guid
Render	CommandBuffer, IMaterial, Renderable sort OpenGL* + NanoVG (ENGINE_WITH_WINDOW)
UI	UiDocument, Stylesheet, ViewModel, UiCanvas
Audio	IAudioSystem / AudioSystem / FakeMixer
Haptics	IHaptics / HapticsSystem
Tools	asset_codegen, asset_guid, icon_codegen
Platform	SDL3, glad, NanoVG, glm, spdlog, tinyclang, tomlplusplus, boost.di

4.2. Граф залежностей (намір vs факт)

Гра ->	IGame, World, AssetsDb, UI VM, Input actions, IAudioSystem, IWindowControl
Гра ->	SDL, glad, NanoVG, spdlog, tinyclang (форсується include-шляхами)
Engine<GameT> ->	EngineRuntime, DI, AssetsDb, register_engine_systems
EngineRuntime ->	WindowManager -> WindowSystem, OpenGLCanvas, OpenGLRenderBackend
run_render ->	CommandBuffer, Camera, Transform, Renderable, ui::WindowSize
run_ui_render ->	UiCanvas, CommandBuffer (на WindowId)
AssetsDb ->	IGraphicFactory (опційно; NotReady без нього)
Camera ->	ui::WindowSize (інверсія)
Command ->	ui::UiDocument* (інверсія)
systems.cpp ->	майже кожен модуль (хаб)

Ігри фізично не можуть інклюдити OpenGL/NanoVG/spdlog — це найсильніше рішення модульності в репозиторії.

4.3. Картка підсистем

Підсистема	Відповідальність	Залежить від	Від кого залежить	Володіє даними	Публічне API	Незалежно reusable?	Прив'язана до поточної гри?
Core host	Init, цикл, платформа	SDL, World, усі сервіси	Гра	Runtime, вікна	Engine, Host, IGame	Host — так; Engine — ні	Overlay-шлях — так
ECS World	Сутності, views, системи, ctx	Нічого engine-specific	Усе	Pools, resources	World, Entity, Events	Так	Ні
Schedule	Фази Fixed/Frame	Закритий enum	Ігри + системи рушія	Списки функцій на World	Schedule/Phase	Слабко — enum глобальний	Пайплайн 2D realtime
Physics	Інтеграція + overlap-події	Transform, Time	Ігри, що читають CollisionEvent	PhysicsOverlapState y ctx	RigidBody, колайдери	Так як код; ні як обов'язкова фаза	Типів немає; зайнятість фази — так
Render	Sort + команди + GL	Assets, Camera, UI painter	Present	GPU через factory	Commands, materials, factory	Інтерфейси — так; хост — ні	World draw лише в primary

Підсистема	Відповідальність	Залежить від	Від кого залежить	Володіє даними	Публічне API	Незалежно reusable?	Прив'язана до поточної гри?
UI	Розмітка, layout, MVVM, hit-test	World, Assets, painter	Input, UiRender	Клонований UiInstance	UiCanvas, ViewModel	Парсер — так; runtime через World	Overlay hit-test / MouseConsumed
Assets	GUID-каталог, sync load, кеш	Файлова система, factory	Майже все	Кеш shared_ptr<void>	AssetsDb, AssetId	Так	Ні
Audio	Шини, SFX-пул, музика A/B	Mixer або fake	Події з гри/UI	Треки в Impl	IAudioSystem	Так	Політика Lumenwake
Haptics	One-shot вібрація	Platform ifdefs	Гра	Impl	IHaptics	Так	Ні
Input	Іменовані action, миша/тач	Черги подій World	Гра, UI	Таблиця bind	InputSystem, ActionId	Здебільшого	Геймпад не імплементовано

4.4. Гарячі точки зв'язаності

- **Високий fan-in:** `ecs::World, AssetsDb, AssetId, Time`
- **Високий fan-out:** `EngineRuntime, register_engine_systems / systems.cpp, Engine<GameT>::init`
- **God-класи:** `EngineRuntime, WindowSystem` (GL-вікно + overlay + hit-test + іконки)
- **Випадковий locator:** `World::ctx<T>()`
- **Майже-цикли компіляції:** `camera.h → ui/canvas.h; commands.h → UI; igrade.h → world.h + window_desc.h + builtin_ids.h`
- **Класичного циклу А→В→С→А в заголовках не знайдено.** Справжнє коло — `*runtime*`: системи `World` захоплюють вказівники `Engine`, який володіє грою, яка володіє `World`.

5. Результати сценаріїв змін

Сценарій	Поточна архітектура	Основні системи	Радіус ураження	Архітектурний блокер
А — Інша 2D-гра (нові сутності, цикл, UI)	Нові компоненти і системи <code>Phase::Game</code> працюють. XML UI і GUID-асети перевикористовуються. Доведеться жити з overlay API, завжди увімкненими physics/UI, дефолтним splash Wind, фіксованим кроком 60 Гц	Здебільшого репозиторій гри; рушій — якщо цикл/фази інші	Малий-середній для іншої 2D action/HUD гри; середній , якщо цикл не Fixed+Frame	Закритий Phase; overlay-подібний <code>IWindowControl</code> ; немає scene graph, якщо потрібна ієрархія (задокументований ліміт v1, прийнятно)
В — Інший рендерер	Інтерфейси є, але хост/бекенд/матеріали/шейдери/UI painter — <code>GL+NanoVG.dynamic_pointer_cast<OpenGL*></code>	<code>EngineRuntime, WindowManager, OpenGL*, NanoVgPainter, shader_adapt, матеріали, preload шрифтів/картинок в Engine::init</code>	Великий	Хост конструює OpenGL-типи; порожні GPU-інтерфейси; модель uniform/матеріалу; UI painter прив'язаний до картинок NanoVG

Сценарій	Поточна архітектура	Основні системи	Радіус ураження	Архітектурний блокер
C — Мультиплеер	Генераційний Entity локальний. Немає серіалізації, net-id, біта authority, checksum. Події — кільцевий буфер на 2 покоління. Порядок overlap фізики сортований за Entity (достатньо стабільний для старту, не мережева модель)	World, events, physics, assets, будь-який геймплей	Великий, але не заблокований типами	Немає snapshot/delta; синглтони ctx; shared_ptr GPU змішаний із sim-компонентами; інпут прив'язаний до пристрою
D — Dedicated server	ENGINE_WITH_WINDOW=OFF + Host + FakeCanvas можуть тикати сим без GL/аудіо. Engine<GameT> не може: вимагає video, вікно, UI-шрифт, preload текстур, init haptics. engine_add_game вимагає ENGINE_WITH_WINDOW	Розкол Host vs Engine; GPU upload каталогу на init; Input poll	Середній	Віконний Engine — не sim-хост; Host не політь серверний годинник/мережу; UI/audio системи все одно реєструються
E — Багатопотоковість	SDD і код: лише main thread. Немає mutex на World/Assets/GL. std::function системи, sparse ECS, ctx-глобали, reentrant tick уже повторно входить у той самий світ	World, AssetsDb, GL, audio	Перепис World + host, щоб вийти за межі «джоби, що не чіпають API рушія»	Спільний мутабельний World; відкладений destroy через view_depth однопоточковий; Boost.Dl shared-синглтони
F — Стрімінг світу	Один World, один каталог, нескінченний кеш, init заливає всі UI-картинки, reuse індексів Entity, немає чанків, на компонентах немає хендлів ресурсів окрім AssetId (a Renderable тримає живі GPU ptr)	AssetsDb, Engine init, Renderable, World	Великий	Жадібний GPU upload; немає unload; ownership Renderable; немає об'єкта scene/region
G — Моді / скрипти	Контент: GUID-каталог + XML/CSS + try_get — справжній шов. Логіка: C++ системи, codegen asset_ids.h на збірці, немає VM, ViewModel — C++	Assets, UI, codegen, бінарник гри	Середній для контенту; великий для логіки	Немає межі script/native plugin; fatal get для відсутніх асетів; колізії intern BindingId валять cook

6. Архітектурні сильні сторони

Це треба зберегти. Воно реально працює.

- 1. Публічний/приватний корінь інклюдів** (CMakeLists.txt: PUBLIC include/, PRIVATE src/). Ігри фізично не можуть підключити OpenGL/NanoVG/spdlog. Це найсильніше рішення модульності. Не кладіть заголовки «please don't use» у include/engine/detail/.
- 2. IGame / GameBase як контракт engine/game.** Гра володіє World. Рушій реєструє свої системи; ігри додають Phase::Game. Розкол ясний і задокументований, Host це тестує (RegisterEngineSystemsBeforeOnStart).
- 3. Command buffer: лише** CmdDrawMesh / CmdDrawUI. Немає CmdCustomDraw, немає GL-колбеків із коду гри. Правильне «один спосіб малювати» для малого рушія. Тримайте набір закритим.
- 4. GUID-асети + cooked-каталог + get/try_get.** Імена файлів — не runtime API. Налаштування імпортера живуть у .meta. Це переживає перейменування, пакування і (пізніше) моди краще, ніж завантаження за шляхом.
- 5. Headless-by-default репозиторій рушія.** ENGINE_WITH_WINDOW/ENGINE_WITH_AUDIO вимкнені в корені; Host + fake; engine_tests ніколи не бутить Engine::run. Саме тому тестованість висока. Не починайте вимагати GPU як merge gate.
- 6. Події у стилі Bevy Events<T> замість callback bus.** Подвійний буфер + явний EventCursor прості й уникають багів lifetime спостерігачів. Залиште.
- 7. Іменованний інпут** ActionId, а не сирі клавіші в геймплей-системах. Миша все одно емітить MouseEvent для UI. Правильний розкол.
- 8. Pimpl на** EngineRuntime, AudioSystem, HapticsSystem, Audio. Mixer/GL/JNI лишаються поза публічними заголовками.
- 9. UI як розмітка + VM, не дерева** onClick у C++ гри. ICommand / RelayCommand — справжня межа UI→гра. Codegen BindingId з XML — гарна перевірка на етапі збірки.
- 10. WindowManager тримає** unique_ptr **слотів**, щоб посилання OpenGLCanvas не звисали при rehash. Хтось думає про lifetime, не лише про фічі.
- 11. Фіксований крок із pause, який не зсипає 8 симуляційних тиків** (FixedStepClock::advance). Frame усе одно біжить для pause UI. Зрілий вибір циклу; збережіть.
- 12. Явні non-goals v1 у SDD** (немає мережі, редактора, parent трансформів, Vox2D). Проект не прикидається Unreal. Не «доповнюйте» це спекулятивними фреймворками.

7. Дорожня карта рефакторингу

Fix Now

Структурні проблеми, які дорожчають, щойно більше ігор закріплять цей коміт.

- 1. Припинити заливати кожен texture/font каталогу в** Engine::init. Вантажити on demand через AssetsDb::get. Локальна зміна, пізніше розблоковує стрімінг/DLC.
- 2. Узгодити IGame::on_draw із реальністю.** Або викликати з Host і EngineRuntime перед present, або прибрати. Тести зараз документують, що воно мертве (Host.FakeCanvasDraw).
- 3. Одна функція tick** між Host і EngineRuntime::tick_loop (present/poll як ін'єктовані порти). Не дозволяйте overlay-реентрантності винаходити третю копію без спільного ядра.
- 4. Не нарощувати overlay-поведінку всередині tick_loop.** Наступний фікс click-through/drag має йти за політикою вікна, інакше витягти це вже не вдасться.

Fix When Touching This System

- 1. register_engine_systems opt-in** (physics/UI/audio/render окремо), коли наступного разу змінюватимете schedules.

2. **WindowSize у core; одна мапа на WindowId**, коли наступного разу чіпатимете multi-window.
3. **Renderable зберігає AssetId (або хендли), не shared_ptr<IMesh>**, коли чіпатимете render-систему.
4. **Камера перестає інклудити UI-заголовки**; передавати size-struct із core.
5. **Fatal-шлях**: report потім quit, не report потім abort, коли чіпатимете AssetsDb.
6. **Геймпад**: імплементувати або прибрати значення ControlKind, коли чіпатимете input.
7. **Винести SplashTimer з Phase::Input**, коли чіпатимете splash або фази.

Don't Fix

Технічно недосконало, не варто окремого перепису.

- Самописний ECS у формі EnTT. Відповідає SDD, тести є, заміна не лікує проблем overlay/host.
- Boost.DI на десяток bind. Дивно (boost/di.hpp у публічному include, коли вікно увімкнене), але це задокументована історія constructor-injection для GameT. Міняти на ручний make_shared — смак, доки DI не почне розповзатися.
- Порожні IMesh/IShader/ITexture як теги типів. Нешкідливо, якщо GL лишається єдиним бекендом.
- AABV-фізика без резолюції. Задокументовано. Не вигадуйте фізичний рушій.
- Відсутність ієрархії трансформів. Задокументований v1.
- NanoVG + обмежений CSS. Це і *є* UI-продукт.
- Системи як std::function. Нормально на цьому масштабі.
- kFixed = 1/60. Константа, не архітектурна пастка.
- Віртуальні IAudioSystem / IHaptics з однією прод-імплементациєю + тестовим fake. Fake — друга імплементация; віртуали себе відпрацьовують.

Future Considerations

- Справжня заміна рендерера (лише якщо другий GPU API запланований).
- Unload асетів / стрімінг / async load.
- Окремий SimHost без ICanvas.
- Сериалізація і мережеві ID (не додавати, доки гра не потребує).
- Скриптова VM (XML UI + C++ VM достатньо, доки гра не потребує модів логіки).
- Compile-time розкол на engine_core / engine_render static libs (лише якщо час збірки або лінковка сервера стануть реальним боєм).
- Використання Time::alpha для інтерполяції рендера (поле є, не використовується).

8. Аудит абстракцій (вибірково)

Абстракція	Клас	Нотатки
IGame / GameBase	GOOD	Справжня межа engine/game. on_draw — дірка в контракті.
ecs::World	GOOD	Ясна відповідальність. ctx робить її LEAKY як locator.
Schedule / Phase	TOO SPECIFIC	Кодує пайплайн цього продукту як єдиний можливий.
Events<T>	GOOD	Просто, явно, тестовано.

Абстракція	Клас	Нотатки
CommandBuffer	GOOD	Закритий набір команд. Вказівники CmdDrawUI роблять його LEAKY .
IRenderBackend / IGraphicFactory	LEAKY / TOO GENERIC	Виглядає змінним; хост і dynamic_cast кажуть інакше. Порожні GPU-інтерфейси майже нічого не дають.
IMaterial	TOO SPECIFIC	Один albedo, три blend, GL-конвенція uniform. Нормально як 2D-матеріал, не як загальна система матеріалів.
AssetsDb	GOOD	GUID-ідентичність правильна. Відсутній unload — MISSING можливість, не погана абстракція.
IAudioSystem	GOOD	Fake міхер виправдовує інтерфейс. Розміри пулів — політика, не абстракція.
IHaptics	GOOD	Платформенні бекенди сховані; fake-стан для тестів.
IWindowControl	TOO SPECIFIC	Змішує звичайні операції з вікном і overlay click-through/drag.
ViewModel / ICommand	GOOD	Справжня межа UI/гра.
UiDocument Element	LEAKY	Весь layout AST публічний; ігри можуть мутувати visual tree попри SDD.
Граф Boost.DI	TOO GENERIC	Один call site; публічна залежність; скрізь shared_ptr. Цінна частина — constructor injection GameT.
Host	GOOD	Справжній шов тестів/сима.
Фізичні компоненти	TOO SPECIFIC як «фаза фізики»; GOOD як опційний 2D overlap	Залишити код; не залишати обов'язковим.
Синглтон ActiveCamera	TOO SPECIFIC	Одна ortho-камера на primary window.

9. Ownership у C++ і потік даних

Хто чим володіє

Об'єкт	Власник	Lifetime	Ризик
EngineRuntime / вікна / GL	Engine, unique pimpl	init→dispose	Closure систем можуть переживати dispose, якщо World збережено
ecs::World	IGame / GameBase	Об'єкт гри	Захоплені deps систем рушія звисають, якщо Engine помирає першим
GPU mesh/shader/texture	Кеш AssetsDb shared_ptr + копії на Renderable	Процес; ніколи не виселяється	Зайві ref подовжують життя GPU; немає unload
Клон документа UiInstance	Компонент сутності	До rebuild canvas	Raw ptr CmdDrawUI лише same-frame
ViewModel	Ігровий shared_ptr на UiCanvas	Контролює гра	Raw ptr UiInstance::loaded_data_context

Об'єкт	Власник	Lifetime	Ризик
InputSystem::world_	Non-owning	Має збігатися з поточним світом гри	Встановлюється в Engine::init після конструкції гри
Logger spdlog	Function-local static	Процес	Єдиний помітний global; прийнятно

Потік даних (один кадр, віконний режим)

1. flush_events старить Events<T>
2. SDL poll → InputSystem пише InputEvent/MouseEvent
3. Overlay може синтезувати mouse move з OS-курсора
4. FixedStepClock → 0..8 × on_fixed_update → Schedule::Fixed (physics, game)
5. on_update → Schedule::Frame: UI input/splash → **гра** → bind → audio events → CmdDrawMesh → CmdDrawUI
6. update_click_through(MouseConsumed), потім draw_all (бекенд виконує команди)

Стан симуляції живе в ECS-компонентах. Стан рендера перебудовується щокадру (добре). Visual tree UI — клонований документ, який мутують у Bind (побічний ефект: hover/press/layout_rect на Element). Аудіо витягується подіями, потім ним володіє міксер. Бар'єра запису між sim і render немає.

10. Модель виконання

Ініціалізація (віконна): init_video → log → DI конструює сервіси+гру → create_window → init audio/haptics → каталоги → **усі шрифти/картинки** → іконка → register_engine_systems → apply_canvas_fit. Порядок неявний і навантажений (вікно перед GL upload шрифту; каталог перед get).

Вимкнення: dispose audio/haptics, runtime_.shutdown (WindowManager + SDL_Quit). on_quit іде з end_loop / деструктора Host, не з самого Engine::dispose. Engine::run фактично кличе dispose двічі (run кличе dispose після runtime_.run, який уже завершив shutdown). Нешкідливо, якщо є охорона initialized_, але легко помилитись.

Припущення про порядок (реальні, частина неявна)

- Physics перед Fixed Game (явний enum)
- Input перед Frame Game, щоб MouseConsumed було видно (явно; overlay залежить від цього)
- Game перед Audio, щоб SFX того ж кадру працювали (явно, задокументовано)
- Render перед UiRender, щоб HUD був зверху (явно)
- run_render чистить primary command buffer; run_ui_render сам чистить вторинні буфери
- Host і run_input обидва кличуть ui::begin_frame (випадковий подвійний reset, якщо біжать разом — а вони біжать)

reentrant_tick документує, чому не можна кликати flush_events. Цей коментар — доказ, що модель виконання вже надто тонка.

11. Модулі та компіляція

- Одна STATIC-бібліотека engine; file(GLOB_RECURSE src/*.cpp із фільтром OpenGL/runtime, якщо немає ENGINE_WITH_WINDOW.
- Логічні папки (core, ecs, render, ui, ...) **не** є CMake-таргетами. Будь-який .cpp може інcludити будь-який приватний заголовок.

- `include/engine/engine.h` — парасолька, що тягне весь публічний API, включно з UI та аудіо.
- Реалізація `World` у публічному `world.inl` — кожна TU гри, що інклудить `world.h`, інстанціює пули.
- `Engine<GameT>` header-only і інклудить `<boost/di.hpp>` у віконному режимі, тож **translation unit гри** компілює `Boost.DI`.
- `engine_instantiate.cpp` явно інстанціює `Engine<WindowSmokeGame>` — compile-time smoke, не межа модуля.
- Тести приватно інклудять `src/` — прийнятно для тестів і було б порушенням межі, якби так робили ігри.

Така складність доречна для малого рушія **доти**, доки `dedicated server` не захоче лінкуватись без `NanoVG/UI`, або поки час заголовків не заболить. Тоді дробити таргети; не дробити заради чистоти.

12. Дизайн API (погляд програміста гри)

Легко використати добре: `GameBase`, `world.create/emplace/view`, `EventWriter/EventReader`, `AssetsDb::get<T>(builtin::mesh_quad)`, `intern("jump") + bind(KeyCode::Space, id)`, `XML {binding} + RelayCommand`.

Треба розуміти нутроші:

- `MouseConsumed` треба перевіряти в `Phase::Game`, інакше `action`, прив'язані до миші, пробивають UI (задокументовано, легко пропустити).
- `EventReader` без `EventCursor` повторно віддає події два кадри.
- `World::get` — `assert`; `AssetsDb::get` — діалог і `abort`.
- Розмір вторинного вікна — не `ctx<WindowSize>()`.
- `IWindowControl::set_drag_region` краде кліки в кнопку усередині прямокутника (задокументований WARNING).
- `Renderable` потребує вказівників `mesh+material`, інакше `render fatal`.
- `ActiveCamera.entity` має бути валідним, інакше `world draw` пропускається (UI може малюватись далі).
- Ін'єктовані сервіси через конструктор `GameT` лише якщо ви в DI; тести `Host` конструюють ігри без DI.

`ICanvas` — `virtual void draw()`: викликач не може сабмітити команди через нього. Команди — побічний канал із ECS-систем. Це ОК, якщо ігри самі не малюють; це робить `on_draw` ще безглуздішим.

13. Ресурси, продуктивність, тестованість, розширюваність

Ресурси: `Sync load`, кеш на життя процесу, ідентичність = GUID, немає `hot reload (non-goal)`, немає `async`. `Cook` на збірці (`asset_codegen`). Добре відомі `builtin GUID` не можна регенерувати. Це міцний v1 **якщо** `init` перестане `preload`-ити світ.

Продуктивнісна архітектура

- **Фактична проблема:** `run_physics` — $O(n^2)$ пари; `run_render` копіює кожен `Renderable` (два `shared_ptr`) у вектор для `sort`; один GL `draw` на `mesh`, без батчінгу; `std::function` на систему; `overlay` може політь курсор щодакдру; усі UI-картинки в `NanoVG` на старті.
- **Майбутня:** Це стане важливим на сотнях колайдерів / тисячах спрайтів. Це не сьогоднішній баг. Не робіть «ECS, але data-oriented» перепис без профілю.

Тестованість: Найвища оцінка. ECS, події, фізика, математика камери, intern інпуту, парсинг UI/MVVM, матеріали, sort, аудіо-політика, haptics, splash, цикл host, хелпери платформи покриті GoogleTest. Є fake для canvas, mixer, haptics, fatal error. Слабо: немає тесту Engine::run, немає GPU golden (явний non-goal), Host не підключає EngineSystemDeps.

Розширюваність

Додавання	Без правок ядра?
Новий геймплей-компонент/система	Так (Phase::Game)
Новий XML-віджет	Hi (enum + парсер + painter) — розумно
Новий імпортер асетів	Hi (typeid-switch у AssetsDb + ImporterKind) — розумно
Новий аудіо-бекенд	Здебільшого (IAudioSystem); CMake все ще SDL-подібний
Новий пристрій інпуту	Hi (EngineRuntime::poll_events + InputSystem)
Новий render-бекенд	Hi — див. проблему 2.3
Новий physics-бекенд	Не можна реєструвати run_physics, інакше біжать обидва
Новий toolkit вікон	Hi — SDL і є платформа

14. Перинженерія vs недоінженерія

Перинженерія: Boost.DI як публічний граф на ~10 bind; GPU IMesh/IShader/ITexture без методів; значення геймпада в ControlKind; подвійні мапи WindowSize заради call site; коментарі overlay-реентрантності довші за Host::tick.

Недоінженерія (хороша): Один World, немає scene graph, немає job system, немає рефлексії, немає plugin API, лише WAV, AABB overlap.

Недоінженерія (шкідлива): Немає on-demand GPU load; немає опційних систем рушія; немає спільного tick; заміна рендерера фіктивна; on_draw мертвий.

Складність майже доречна. Same overlay-набір фіч виштовхнув хост за межі «малого 2D-рушія» в «Windows overlay framework».

15. Радіус ураження (вимога → код)

Вимога: «прозорий overlay має пропускати кліки в порожньому місці» → MouseConsumed + UI hit-test → WindowSystem::update_click_through / Win32 subclass → Полінг курсора в tick_loop, коли motion-події зупиняються → Конфлікт drag-region із HTTRANSPARENT → Reentrant tick, щоб drag не заморожував сим

Продуктова вимога однієї гри змусила зміни в UI, віконності, головному циклі та flush подій. Це визначення великого радіуса ураження.

Вимога: «додати вікно налаштувань» → WindowId, WindowManager, подвійні мапи WindowSizes/UiPointers, commands_for_window, replay шрифтів, close-події. World render усе ще лише primary. Середньо-великий радіус, і подвійні мапи — шрам, що лишився.

Вимога: «відтворити звук» → PlaySfxEvent → фаза Audio → AssetsDb::get<Sound> → міксер. Малий радіус. Це здорово.

Вимога: «змінити крок часу» → Константи kFixed у публічному заголовку. Малий радіус.

16. Найважливіше архітектурне питання

Якщо я розвиватиму цей рушій ще 2–3 роки, про які 3 архітектурні рішення я найбільше пошкодую, що не виправив зараз?

1. Desktop-overlay віконність як дефолтний runtime рушія. Click-through, drag-регіони, Win32 subclassing, синтетичний полінг миші і `reentrant_tick` тепер визначають, що таке «кадр» (`src/core/engine_runtime.cpp`, `src/render/opengl/window_system.cpp`, `IWindowControl`). Кожна майбутня гра і платформа тягнутиме цю історію, доки overlay не стане політикою поверх нудного циклу.

2. Жадібні безсмертні ресурси плюс `Renderable`, що тримає живі GPU `shared_ptr`. `Engine::init` заливає кожен шрифт і UI-картинку; `AssetsDb` ніколи не виселяє; `drawable` ділять володіння GL-об'єктами. Це тихо перетворює стрімінг, великі каталоги і навіть «завантажити рівень, вивантажити рівень» на перепис замість додавання API.

3. Закритий `enum Phase`, обов'язковий `register_engine_systems` і `World::ctx<T>()` як модель композиції. Пайплайн не змінити без правок заголовків рушія; `physics/UI` біжать навіть коли `deps null`; новий наскрізний стан стає ще одним ctx-синглтоном (`WindowSizes`, `SplashTimer` в `Input`). За два роки фічі зваляться в `systems.cpp` і `ctx`, доки рушій не стане рантаймом конкретної гри з зайвими кроками.

Інтерфейси, які треба берегти: `IGame`, `command buffer`, `GUID AssetsDb`, `Host` як headless-шов, події замість колбеків, стіна `public/private include`. Інтерфейси, яким не варто довіряти: «ми можемо замінити рендерер, бо є `IRenderBackend`» і «ми overlay-агностичні, бо click-through сховано за `IWindowControl`».